IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 13 January 2026, accepted 9 February 2026, date of publication 23 February 2026, date of current version 27 February 2026.

*Digital Object Identifier 10.1109/ACCESS.2026.3667131*

RESEARCH ARTICLE

# Distilling Privileged Knowledge From Transformers to Lightweight CNNs for On-Device Time Series Forecasting

**SANGJIN NA, YONGHUN CHO, AND YUNJU BAEK, (Member, IEEE)**
School of Computer Science and Engineering, Pusan National University, Busan 46241, South Korea

Corresponding author: Yunju Baek (yunju@pusan.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP)-Information Technology Research Center (ITRC) grant funded by Korean Government of the Ministry of Science and Information and Communication Technology (ICT) under Grant IITP-2026-RS-2023-00260098; and in part by IITP through the Artificial Intelligence Convergence Innovation Human Resources Development grant funded by Korean Government (MSIT) under Grant IITP-2026-RS-2023-00254177.

**ABSTRACT** Time Series Forecasting (TSF) is a pivotal capability for critical applications in industrial automation, energy management, and smart infrastructure. While Large Language Models (LLMs) and Transformer-based architectures have set new benchmarks in forecasting accuracy, their exorbitant computational costs and memory requirements prohibit their deployment on resource-constrained Microcontroller Units (MCUs). Lightweight models offer an alternative but often fail to capture complex long-term dependencies. This paper introduces a comprehensive framework to address these interconnected challenges of accuracy and efficiency. We propose a highly efficient Convolutional Neural Network (CNN) architecture, enhanced through a novel heterogeneous knowledge distillation methodology. In our framework, a powerful LLM-based teacher model, utilizing self-attention mechanisms, guides the training of a compact student model that operates solely on lightweight convolutions during inference. This approach is augmented by the Gated Dilated depthwise separable Convolution (GDC) block, which efficiently captures long-range dependencies and multi-scale patterns while drastically minimizing parameter count to meet strict MCU constraints. The robustness and generalizability of our framework are validated through extensive experiments on six long-term and five short-term forecasting benchmarks. Critically, our framework establishes a new state-of-the-art among lightweight architectures, demonstrating consistent performance gains across diverse forecasting horizons. Experimental results confirm that our method reduces Mean Squared Error (MSE) by 11.5% in long-term and 12.4% in short-term forecasting tasks compared to massive LLM-based baselines, all while utilizing less than 0.01% of the parameters. The final, optimized model is successfully deployed on a commercial ESP32-S3 MCU, demonstrating its practical viability for real-time, accurate, and efficient forecasting on edge devices with an inference latency of just 256 ms.

**INDEX TERMS** Time series forecasting, knowledge distillation, heterogeneous knowledge distillation, lightweight convolutional neural networks, model compression, TinyML, edge AI.

## I. INTRODUCTION

Time Series Forecasting (TSF) is a core technology for ensuring operational efficiency and reliability in modern industries, with applications such as traffic flow prediction [1], electricity demand management [2], and predictive maintenance in smart factories [3]. The widespread deployment of Internet of Things (IoT) devices and industrial sensors has resulted in exponentially growing data volumes in edge environments [4]. Traditionally, these data have been transmitted to centralized cloud servers for analysis. However, limitations in network bandwidth, transmission latency that hinders

The associate editor coordinating the review of this manuscript and approving it for publication was Ganesh Naik.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

real-time responsiveness, and security and privacy concerns associated with sensitive data have driven a sharp increase in demand for on-device AI, where inference is performed directly on the data-generating devices. This shift is more than a change in where computation occurs, introducing new technical challenges as high-performance inference must be achieved under stringent resource constraints [5].

Recent advances in deep learning have substantially improved the accuracy of TSF. In particular, there is growing interest in adapting Transformer architectures and Large Language Models (LLMs) to time series data. Recent methods such as AutoTimes [6] and Time-LLM [7] leverage large-scale pretraining and strong contextual modeling capabilities to better capture long-term dependencies in time series, achieving state-of-the-art forecasting performance. Despite these improvements, they remain fundamentally limited by their reliance on extremely large parameter counts and substantial computational demands [8]. Such requirements are incompatible with edge platforms like Microcontroller Units (MCUs), where available SRAM and non-volatile memory capacity are severely constrained. Consequently, the gap between model complexity and device-level resource constraints continues to hinder the practical deployment of advanced TSF methods in real-world industrial on-device AI systems.

To address the trade-off between accuracy and efficiency, lightweight architectures such as DLinear [9] have been proposed. However, their inherent linearity limits their ability to capture complex non-linear dynamics and volatile temporal patterns. To overcome this limitation, recent studies, exemplified by TimeKD [10], have adopted Knowledge Distillation (KD) to compress the predictive capabilities of high-capacity models into smaller ones. However, these existing methods focus primarily on homogeneous distillation (i.e., Transformer-to-Transformer), where the student model retains a Transformer-based architecture. Consequently, the student inherits the quadratic computational complexity ($O(L^2)$) of the self-attention mechanism [11], rendering it unsuitable for real-time inference on severely resource-constrained devices such as MCUs. To overcome this, recent research in computer vision has demonstrated the potential of heterogeneous distillation to bridge the gap between different architectures [12].

In this work, we propose several approaches to mitigate the critical trade-off between forecasting accuracy and computational efficiency in TSF. First, we introduce a novel heterogeneous knowledge distillation strategy that transfers the strong representational capacity of a Transformer-based teacher to a lightweight Convolutional Neural Network-based (CNN-based) student. To address the feature-space mismatch induced by architectural heterogeneity, we design an attention-guided feature distillation loss that leverages the teacher's attention maps to encourage the student to focus on temporally salient regions. Second, we develop a student architecture optimized for the constraints of edge devices. Specifically, we propose a lightweight convolution block that integrates dilated convolutions [13] to expand the receptive field, Depthwise Separable Convolutions (DSC) [14] to improve parameter efficiency, and Gated Linear Units (GLU) [15] to enable dynamic feature selection. Finally, to demonstrate the practicality and robustness of the proposed method, we extend the evaluation beyond long-term forecasting to include short-term forecasting tasks, and we assess generalization performance under diverse benchmark datasets. We further validate real-time on-device inference with low memory footprint by deploying the distilled model on an ESP32-S3 MCU. The main contributions of this study are summarized as follows:

- We propose a heterogeneous knowledge distillation framework that effectively bridges the gap between a large LLM-based teacher and a lightweight CNN-based student.
- We design a highly efficient CNN architecture that captures long-range temporal dependencies with a minimal parameter budget, thereby optimizing deployment in edge computing scenarios.
- We introduce a novel distillation loss that exploits the teacher's attention mechanism to emphasize salient temporal patterns and enhance the student's feature representations.
- We demonstrate state-of-the-art performance on both long-term and short-term forecasting tasks, and further validate the feasibility of real-time inference through deployment on an actual MCU device.

## II. RELATED WORK

### A. DEEP LEARNING-BASED TIME SERIES FORECASTING

Traditional time series forecasting relied heavily on statistical models such as ARIMA [16] and exponential smoothing methods [17]. However, these approaches show limited capacity to capture non-linear and complex patterns in real-world data. With the advancement of deep learning, Recurrent Neural Network (RNN)-based models, such as Long Short-Term Memory (LSTM) [18] and Gated Recurrent Unit (GRU) [19], were widely adopted for sequence modeling. Despite their success, these sequential models suffer from vanishing gradient issues and low computational efficiency when processing long sequences. Consequently, their performance is often constrained in long-term forecasting settings.

Following the success of the Transformer architecture in Natural Language Processing (NLP), numerous studies have investigated its adaptation to TSF. Initial research primarily focused on mitigating the quadratic computational complexity ($O(L^2)$) of the standard self-attention mechanism. For example, Informer [20] introduces ProbSparse attention to improve efficiency over long sequences. Subsequently, Autoformer [21] incorporates a decomposition architecture with an Auto-Correlation mechanism to better capture complex temporal patterns. Building on these advancements,

FEDformer [22] further enhances the modeling of long-range dependencies by performing attention operations within the frequency domain.

More recent research has advanced toward redesigning model structures to better align with the characteristics of time series data and to maximize forecasting accuracy. PatchTST [11] segments time series into patches to capture local semantic information while retaining global context. Furthermore, it incorporates channel independence, a strategy that substantially enhances performance in multivariate forecasting. In contrast, iTransformer [23] adopts a fundamentally different approach by applying the attention mechanism along the variate dimension rather than the temporal dimension, thereby enabling more effective modeling of multivariate correlations.

Most recently, there has been a surge of interest in adapting the generalization capabilities of LLMs, pretrained on massive text corpora, to TSF [8]. Time-LLM [7] reprograms time series inputs into text prompts and leverages the general knowledge embedded in pretrained LLMs. AutoTimes [6] transforms time series into token embeddings and enables an LLM to generate future values in an autoregressive manner. Parallel to these LLM-adaptation approaches, there is a growing trend towards developing Time Series Foundation Models trained natively on large-scale time series datasets [24]. Models such as TimeGPT [25], Lag-Llama [26], Chronos [27], and Moirai [28] leverage vast archives of time series data to achieve zero-shot forecasting capabilities across diverse domains. Although these approaches achieve state-of-the-art predictive accuracy, they face fundamental limitations regarding deployment on resource-constrained edge devices. This limitation is due to the substantial computation and memory demands incurred by models possessing billions of parameters.

### B. LIGHTWEIGHT TIME SERIES FORECASTING

To mitigate the high computational cost and memory requirements of large-scale models, recent research has actively explored lightweight time series forecasting architectures that avoid complex attention mechanisms and instead emphasize structural simplicity. These approaches can be broadly categorized into Multi-Layer Perceptron-based (MLP-based) models and CNN-based models.

First, MLP-based studies originate from the hypothesis that simple linear models may be more effective for time series forecasting than complex Transformer architectures. DLinear is a representative example that decomposes a time series into trend and seasonality components and models each component using a single linear layer. Despite its simplicity, it achieves performance that surpasses several Transformer-based baselines. Building on this idea, TimeMixer [29] introduces a multi-scale mixing strategy within an MLP framework to further improve forecasting accuracy. Furthermore, TiDE [30] demonstrates that a pure MLP-based encoder-decoder architecture can effectively capture long-term dependencies without attention mechanisms. While MLP-based models have the advantages of a small parameter footprint and very fast inference, they are relatively limited in modeling sequential dependencies across time and complex non-linear dynamics. As a result, their performance can degrade when the data exhibit high variability.

In contrast, CNN-based models are well suited for lightweight deployment because they can extract local features with relatively few parameters. TimesNet [31] converts one-dimensional (1D) time series into a 2D representation and applies 2D kernels to capture periodic patterns effectively. SCINet [32] utilizes a recursive down-sampling and convolving strategy to capture temporal dependencies at multiple resolutions. ModernTCN [33] attempts to expand the receptive field by adopting large kernel sizes. However, CNN-based models inherently suffer from a structural limitation in that their receptive field is constrained by fixed kernel sizes. This makes it difficult to capture global context in long sequences. Increasing network depth can partially alleviate this issue, but it reintroduces the drawback of higher computational overhead. Therefore, it is essential to design efficient architectures that preserve the computational efficiency of CNNs while effectively capturing long-range dependencies. Recent works like LightCTS [34] have attempted to improve efficiency by modeling series correlations, yet optimizing for extreme edge constraints remains a challenge.

### C. KNOWLEDGE DISTILLATION FOR TIME SERIES

Knowledge distillation [35] has recently received growing attention as a strategy to achieve both model compactness and strong performance. It is a framework that transfers knowledge from a large teacher model to a lightweight student model. In particular, several studies exploit future ground truth values that are accessible only during training as privileged information for the teacher model, a concept rooted in the Learning Using Privileged Information (LUPI) paradigm [36]. This approach enables the teacher to learn richer representations and to provide more informative guidance to the student model. While advanced strategies such as reinforced distillation [37] have been explored to enhance regression performance, TimeKD [10] remains a representative method that distills knowledge from an LLM-based teacher into a Transformer-based student. However, this line of work has notable limitations. First, the teacher processes time series by converting them into text tokens, which increases the effective input length and reduces inference efficiency. Second, the student model also adopts a Transformer architecture, so the distillation is primarily performed between homogeneous models. As a result, the quadratic computational complexity of attention remains unchanged. This property makes real-time inference difficult on MCU-level edge devices where memory is severely constrained, as highlighted in studies on TinyML [38].

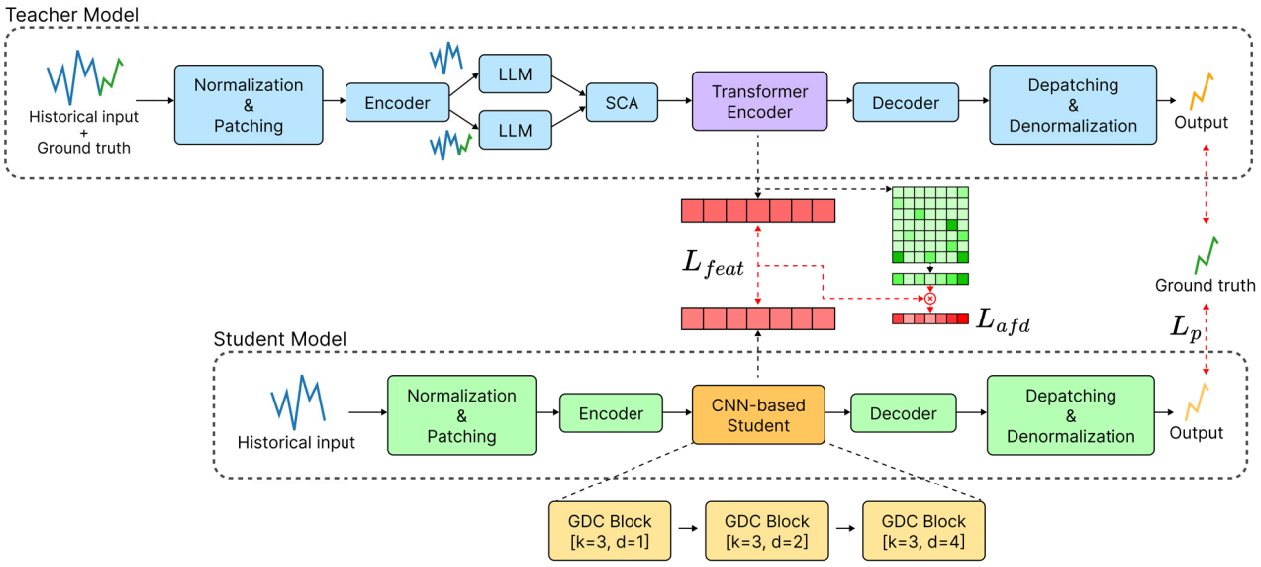



**FIGURE 1. Overview of the proposed heterogeneous knowledge distillation framework. It illustrates the process of distilling knowledge from an LLM-based teacher model to a lightweight CNN-based student model.**

In contrast, this study focuses on knowledge distillation between heterogeneous architectures. We transfer the representational capacity of a strong Transformer-based teacher to a structurally lightweight and efficient CNN-based student. We also process numerical signals directly to eliminate tokenization overhead. In addition, we introduce a new lightweight convolutional block designed to mitigate the limited receptive field of CNNs, enabling high-performance real-time forecasting in MCU deployment settings.

## III. METHODOLOGY

### A. PROBLEM DEFINITION AND PRELIMINARIES

Formally, we address the problem of multivariate TSF. Let $X = \{x_1, \ldots, x_H\} \in \mathbb{R}^{H \times N}$ denote the historical input sequence, where $H$ is the look-back window length and $N$ represents the number of variables. The corresponding ground truth future sequence is denoted as $Y = \{y_1, \ldots, y_T\} \in \mathbb{R}^{T \times N}$, where $T$ is the prediction horizon. Our goal is to learn a mapping function $f_\theta(\cdot)$ that predicts the future sequence $\hat{Y}$ based solely on the historical observations $X$:

$$\hat{Y} = f_\theta(X), \quad \text{where } \hat{Y} \in \mathbb{R}^{T \times N} \tag{1}$$

To mitigate distribution shift and non-stationarity, we apply instance normalization to the input sequence $X$. Given the instance-specific mean $\mu \in \mathbb{R}^{1 \times N}$ and standard deviation $\sigma \in \mathbb{R}^{1 \times N}$, the normalized sequence $\bar{X}$ is computed as:

$$\bar{X} = \frac{X - \mu}{\sigma} \tag{2}$$

The model predictions are subsequently converted back to the original scale using reversible de-normalization with $\mu$ and $\sigma$.

To capture local semantic patterns and reduce computational redundancy, we employ a patching operation. We partition the normalized sequence $\bar{X}$ into non-overlapping patches of length $P$. The number of patches is determined by $L = H/P$. Consequently, following the patching strategy introduced in PatchTST [11], the input is reshaped from a 2D sequence into 3D patches $X_P$:

$$X_P = \text{Patch}(\bar{X}) \in \mathbb{R}^{L \times P \times N} \tag{3}$$

This patched representation $X_P$ serves as the shared input for the encoders of both the teacher and student models described in the subsequent sections.

### B. OVERALL FRAMEWORK

To address the critical trade-off between forecasting accuracy and computational efficiency, we propose a novel heterogeneous knowledge distillation framework. As illustrated in Figure 1, the framework establishes a teacher-student paradigm comprising two distinct architectures. It consists of a high-capacity teacher model leveraging privileged information and a lightweight student model optimized for edge deployment.

The teacher model is built upon an LLM and a Transformer architecture to leverage their capabilities in capturing complex temporal patterns and long-term dependencies. During the training phase, the teacher utilizes privileged information by concatenating historical data with the ground truth future sequence.

In contrast, the student model is designed for real-time inference on resource-constrained devices, such as MCUs. Unlike the teacher, the student relies exclusively on historical observations during both training and inference. To satisfy hardware constraints, we replace the computationally

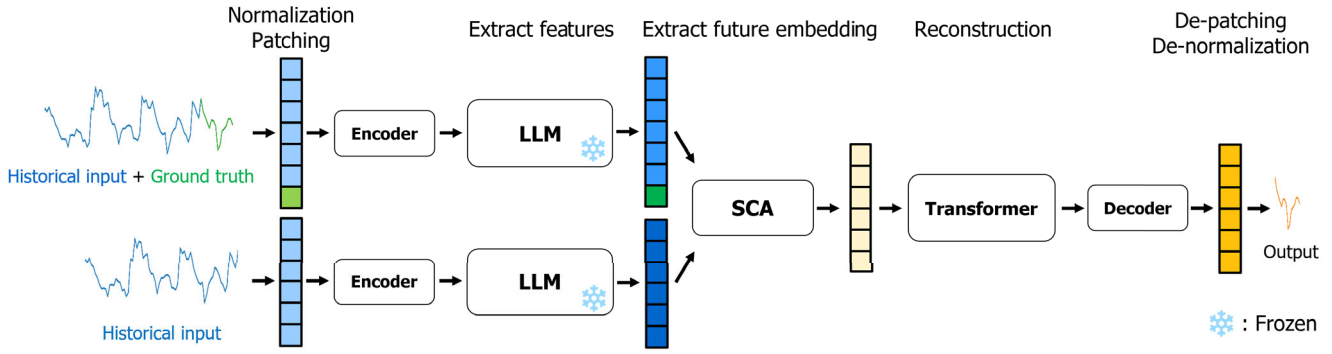



**FIGURE 2.** Overview of the teacher model architecture. The model utilizes a pre-trained LLM to extract latent features from both historical and privileged inputs.

intensive Transformer blocks with a lightweight CNN architecture. Specifically, the student incorporates our proposed Gated Dilated depthwise separable Convolution (GDC) blocks, which efficiently expand the receptive field with minimal parameters, ensuring effective feature extraction under limited resources.

To bridge the architectural gap between the heterogeneous teacher and student models, we employ a two-stage training strategy. The teacher is first pre-trained to extract and store high-quality feature and attention maps. Subsequently, the student is trained to mimic these representations. To enhance this transfer, we introduce a novel Attention-guided Feature Distillation (AFD) loss. This mechanism enables the student to not only mimic the teacher's feature values but also align its focus with the teacher's attention map, effectively identifying significant temporal segments. Consequently, the student inherits the robust predictive capability of the teacher while maintaining a lightweight structure suitable for edge applications.

### C. THE TEACHER MODEL

To facilitate effective knowledge transfer, the teacher model should generate high-quality, expressive feature representations that act as an upper bound for the student's performance. As illustrated in Figure 2, the architecture of the teacher model consists of three key components: an LLM-based embedding layer, a Subtractive Cross-Attention (SCA) module [10], and a Transformer encoder.

#### 1) PRIVILEGED INPUT PROCESSING AND LLM EMBEDDING

The teacher processes two distinct input sequences: the historical sequence $X \in \mathbb{R}^{H\times N}$ and the privileged sequence $X_{priv}$, which is constructed by concatenating the historical data with the ground-truth future data $Y$. Specifically, $X_{priv} = \text{Concat}(X, Y) \in \mathbb{R}^{(H+T)\times N}$. Both sequences undergo the same instance normalization and patching process. The resulting patches are denoted as $P_{hist} \in \mathbb{R}^{L_H\times P\times N}$ and $P_{priv} \in \mathbb{R}^{(L_H+L_T)\times P\times N}$. Here, $L_H = \lfloor H/P \rfloor$ and $L_T = \lfloor T/P \rfloor$ represent the number of patches corresponding to the look-back window and the prediction horizon, respectively, where $P$ denotes the patch length. Subsequently, these patches are linearly projected via a patch encoder to match the hidden embedding dimension $D_{LLM}$ of the pre-trained LLM. To leverage the generalized pattern recognition capabilities of the LLM while mitigating the computational cost of fine-tuning a billion-scale model, we keep the LLM parameters frozen. The patch embeddings are passed through the LLM to yield the historical latent representation $E_{hist}$ and the privileged latent representation $E_{priv}$.

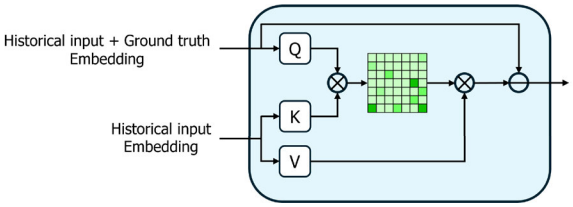



**FIGURE 3.** Structure of the subtractive cross-attention (SCA) module [10]. The module refines the privileged embedding by subtracting the historical information retrieved via cross-attention, thereby extracting pure future features.

#### 2) EXTRACTION OF PURE FUTURE FEATURES VIA SCA MODULE

A naive use of the privileged embedding $E_{priv}$ may lead to information leakage, where the model excessively relies on historical patterns rather than capturing future dynamics. To address this, we adopt the Subtractive Cross-Attention (SCA) [10] mechanism. The primary objective of SCA is to explicitly disentangle and remove the historical information contained within the privileged representation, thereby isolating the pure future features.

As illustrated in Figure 3, the SCA module utilizes the relevant portion of the privileged embedding (corresponding to the prediction horizon) as the query $Q$, and the historical embedding $E_{hist}$ as both key $K$ and value $V$. The refinement process is formulated as:

$$E_{future} = \text{SCA}(Q, K, V) = Q - \text{Attention}(Q, K, V) \quad (4)$$

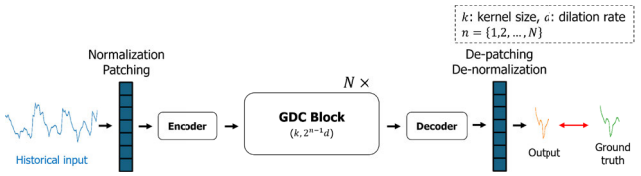



**FIGURE 4.** Architecture of the student model. Designed for efficiency, the student model processes only historical inputs through a stack of Gated Dilated depthwise separable Convolution (GDC) blocks.

where the Attention operation is the standard scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{5}$$

where $d_k$ denotes the dimension of the keys. By subtracting the attention-weighted historical context from the privileged query, $E_{future}$ retains only the information distinct from the past, effectively capturing the novel dynamics inherent in the future sequence.

#### 3) CONTEXTUAL RECONSTRUCTION FOR KNOWLEDGE DISTILLATION

While $E_{future}$ contains the refined future information, the subtraction operation in the SCA module risks compromising the global temporal consistency across patches. To recover this global context and construct a robust representation for distillation, we process $E_{future}$ through a Transformer encoder. This encoder captures the long-range dependencies among the purified future patches, yielding the final teacher feature maps $F_T \in \mathbb{R}^{L\times D\times N}$. These feature maps $F_T$ serve as the high-level knowledge representations that the student model learns to mimic. Finally, a decoder projects $F_T$ back to the original input dimension to compute the reconstruction loss against the ground truth $Y$, ensuring that the features $F_T$ encapsulate predictive features effectively.

### D. THE STUDENT MODEL

While the teacher model prioritizes representation capability, the primary design goal of the student model is computational efficiency and deployment feasibility on resource-constrained edge devices. To mitigate the computational overhead associated with Transformer-based architectures, we adopt a lightweight CNN-based architecture.

As depicted in Figure 4, the student model consists of a projection encoder, a stack of $N$ Gated Dilated depthwise separable Convolution (GDC) blocks, and a decoder. Unlike the teacher, the student processes only the historical patches $P_{hist}$ as input. The encoder first projects these patches into a latent space, after which the subsequent GDC blocks extract temporal features. This design choice ensures robust feature extraction while maximizing parameter efficiency. Finally, the decoder reconstructs the prediction $\hat{Y}_S$ from the extracted features.

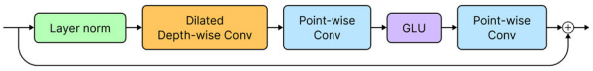



**FIGURE 5.** Structure of the GDC block. The block combines dilated depthwise convolution for an expanded receptive field with a Gated Linear Unit (GLU) for dynamic feature selection.

#### 1) GATED DILATED DEPTHWISE SEPARABLE CONVOLUTION (GDC) BLOCK

Standard CNNs' inherent limitation lies in their restricted receptive fields, which hinders the effective modeling of long-term dependencies. While naively scaling up the kernel size or network depth can mitigate this issue, it incurs a prohibitive computational overhead. To address this trade-off, we propose the GDC block, which synergistically integrates dilated convolutions, DSC, and a gating mechanism to achieve both efficiency and accuracy.

Figure 5 details the architectural components of the GDC block. The operation proceeds as follows: First, the input latent vectors are normalized via layer normalization to stabilize training.

$$X_{norm} = \text{LayerNorm}(X_{in}) \tag{6}$$

Subsequently, to efficiently capture long-range temporal patterns, we employ dilated depthwise convolution [13]. Inspired by MobileNets [14], this operation mitigates computational complexity by performing spatial filtering on each channel independently, while the dilation factor $d$ exponentially expands the receptive field without increasing the number of parameters. This ensures that the model remains parameter-efficient while possessing a sufficiently large receptive field to capture long-term dependencies.

$$X_{dw} = \text{DilatedDWConv}(X_{norm}, d) \tag{7}$$

Following the spatial feature extraction, a pointwise convolution mixes information across the channel dimension. To enhance the model's ability to select relevant features and suppress noise, we incorporate a Gated Linear Unit (GLU) [15], replacing static activation functions like ReLU. This data-driven gating mechanism enables the network to selectively propagate informative temporal signals while attenuating stochastic noise, which is particularly advantageous for modeling non-stationary characteristics in time series data. The GLU dynamically controls the information flow via a sigmoid gate $\sigma(\cdot)$:

$$X_{gate} = \text{GLU}(\text{PWConv}_1(X_{dw})) = A \odot \sigma(B) \tag{8}$$

where $A$ and $B$ are split projections of the pointwise convolution output.

Finally, a second pointwise convolution projects the features back to the original dimension, and a residual connection is added to facilitate gradient flow through deep networks.

$$X_{out} = X_{in} + \text{PWConv}_2(X_{gate}) \tag{9}$$

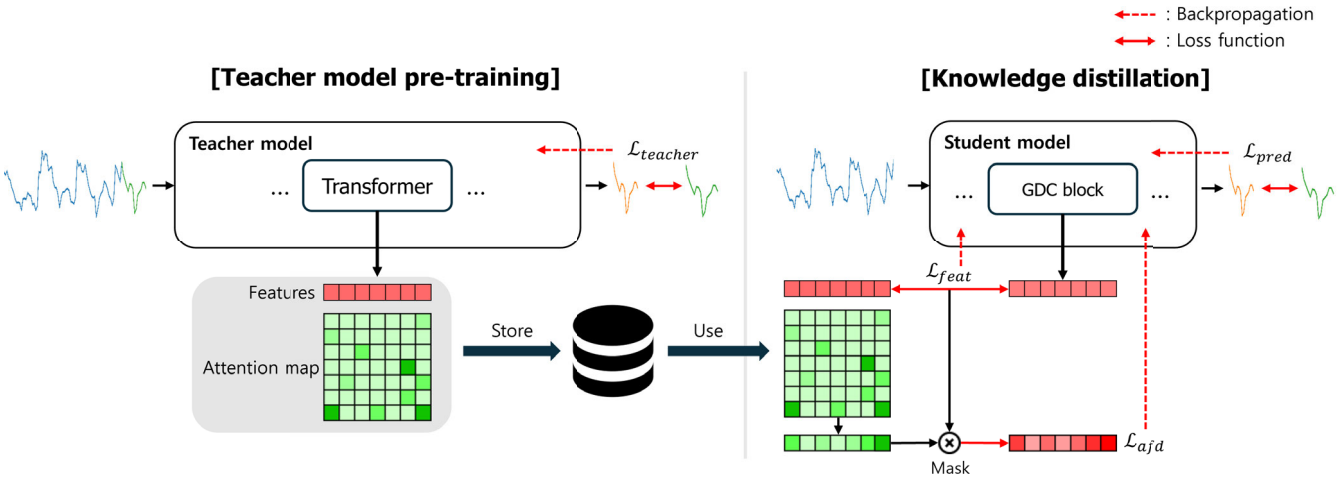



**FIGURE 6.** Overview of the two-stage training framework. In stage 1, the teacher is pre-trained with privileged information to learn ideal representations. In stage 2, the student is trained to minimize both the prediction loss against ground truth and the distillation loss against the teacher's features.

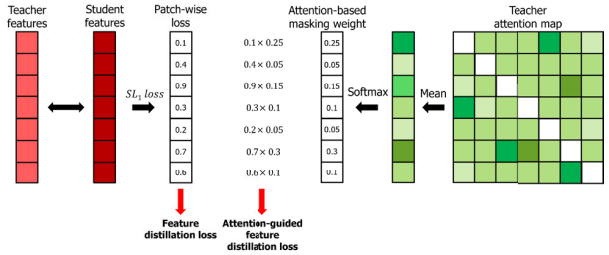



**FIGURE 7.** Illustration of the attention-guided feature distillation (AFD) Loss. The teacher's features are projected to the student's dimension to ensure computational efficiency. The distillation loss is then weighted by the teacher's attention scores, forcing the student to focus on temporally significant patterns.

By stacking these blocks with exponentially increasing dilation rates (e.g., $d = 1, 2, 4, \ldots$), the student model can effectively model multi-scale temporal dependencies with a significantly reduced memory footprint compared to Transformer-based models.

### E. KNOWLEDGE DISTILLATION STRATEGY

Transferring knowledge from a high-capacity Transformer (Teacher) to a lightweight CNN (Student) presents a unique challenge due to the architectural heterogeneity. The two models operate in distinct feature spaces with different channel dimensions and fundamental architectural differences. To bridge this gap effectively, we propose a robust two-stage training framework and a novel Attention-guided Feature Distillation (AFD) loss.

#### 1) TWO-STAGE TRAINING FRAMEWORK

As illustrated in Figure 6, our training pipeline is divided into two distinct stages to ensure the student distills stable and high-quality knowledge.

##### a: STAGE 1: TEACHER PRE-TRAINING

First, following the paradigm of Learning Using Privileged Information (LUPI) [35], the teacher model is trained using both historical data $X$ and privileged ground truth data $Y$. The training objective aims to minimize the discrepancy between the teacher's prediction $\hat{Y}_T$ and the ground truth $Y$. To enhance robustness against outliers prevalent in time series data, we adopt the Huber loss (Smooth L1 loss) instead of the standard Mean Squared Error (MSE):

$$\mathcal{L}_{teacher} = \mathcal{L}_{Huber}(\hat{Y}_T, Y) \tag{10}$$

After the teacher model is fully pre-trained, the teacher's parameters are frozen. We then employ an offline knowledge extraction strategy, storing the intermediate feature maps $F_T$ and attention maps $A_T$ derived from the final layer of the Teacher's Transformer Encoder to reduce computational overhead during the student training phase.

##### b: STAGE 2: STUDENT DISTILLATION

In the second stage, the student model is trained using only historical data $X$. The student is optimized to simultaneously minimize the forecasting error and align its latent features with those of the teacher through a composite loss function comprising three terms: prediction loss, feature distillation loss, and our proposed AFD loss.

#### 2) DISTILLATION OBJECTIVES

##### a: PREDICTION LOSS ($\mathcal{L}_{PRED}$)

The fundamental goal of the student is to accurately forecast the future sequence. We measure the discrepancy between the student's prediction $\hat{Y}_S$ and the ground truth $Y$:

$$\mathcal{L}_{pred} = \mathcal{L}_{Huber}(\hat{Y}_S, Y) \tag{11}$$

##### b: FEATURE DISTILLATION LOSS ($\mathcal{L}_{FEAT}$)

To enable the student to mimic the comprehensive reasoning of the teacher, we enforce consistency between the student's feature map $F_S$ and the teacher's feature map $F_T$. Note that directly computing loss in the high-dimensional space of the LLM teacher is computationally expensive and inefficient.

Therefore, in our implementation, the teacher's features are projected down to the student's feature dimension $D$ via a linear projection layer $\psi(\cdot)$ within the teacher architecture. This ensures that the distillation process remains lightweight and suitable for efficient training. The feature consistency loss is defined as:

$$\mathcal{L}_{feat} = \mathcal{L}_{Huber}(F_S, F_T) \tag{12}$$

where $F_T$ represents the projected features of the teacher, aligned with the student's dimension.

#### c: ATTENTION-GUIDED FEATURE DISTILLATION ($\mathcal{L}_{AFD}$)

While $\mathcal{L}_{feat}$ encourages global feature alignment, it treats all time steps equally. However, in TSF, specific temporal segments often contain more critical information. To address this, we introduce the AFD loss, which leverages the teacher's self-attention mechanism to weigh the distillation loss dynamically.

As depicted in Figure 7, we first derive a temporal importance vector $w \in \mathbb{R}^L$ from the teacher's attention map $A_T \in \mathbb{R}^{H_{heads} \times L \times L}$. We compute the mean attention score across all heads and aggregate the attention weights received by each time step $j$ from all other query positions:

$$w_j = \sum_{i=1}^{L} \left( \frac{1}{H_{heads}} \sum_{h=1}^{H_{heads}} A_{T,i,j}^{(h)} \right) \tag{13}$$

where $A_{T,i,j}^{(h)}$ denotes the attention score from query $i$ to key $j$ in head $h$.

To ensure stable gradient scaling and emphasize relative importance, we apply Z-score normalization before the Softmax operation. The normalized importance scores $\hat{w}$ are computed as:

$$\hat{w} = \frac{w - \mu_w}{\sigma_w + \epsilon}, \quad M = \text{Softmax}(\hat{w}) \tag{14}$$

where $\mu_w$ and $\sigma_w$ represent the mean and standard deviation of $w$, and $\epsilon$ is a small constant for numerical stability.

Finally, the generated mask $M \in \mathbb{R}^L$ is applied to the element-wise feature error, forcing the student to focus on temporally salient regions identified by the teacher:

$$\mathcal{L}_{afd} = \frac{1}{L \cdot D \cdot N} \sum_{i=1}^{L} M_i \sum_{j=1}^{D} \sum_{k=1}^{N} \mathcal{L}_{Huber}(F_{S,i,j,k}, F_{T,i,j,k}) \tag{15}$$

#### 3) CURRICULUM LEARNING STRATEGY

To stabilize the training of the randomly initialized student model, we employ a curriculum learning strategy. Directly imposing strong distillation constraints at the beginning of training can destabilize convergence. Therefore, we dynamically adjust the loss weights using a ramp-up function $r(e)$:

$$r(e) = \min\left(1.0, \frac{e}{E_{warmup}}\right) \tag{16}$$

**TABLE 1. Statistics of long-term time series datasets. We selected six widely used benchmarks covering energy, meteorology, and economics domains.**

| Dataset | Features | Frequency | Samples | Look-back | Horizons |
|---|---|---|---|---|---|
| ETTh1 | 7 | 1 Hour | 17,420 | 672 | {96, 192, 336, 720} |
| ETTh2 | 7 | 1 Hour | 17,420 | 672 | {96, 192, 336, 720} |
| ETTm1 | 7 | 15 Min | 69,680 | 672 | {96, 192, 336, 720} |
| ETTm2 | 7 | 15 Min | 69,680 | 672 | {96, 192, 336, 720} |
| Weather | 21 | 10 Min | 52,696 | 672 | {96, 192, 336, 720} |
| Exchange | 8 | 1 Day | 7,588 | 672 | {96, 192, 336, 720} |

**TABLE 2. Statistics of short-term time series datasets. These datasets consist of electricity prices from five major power markets. Due to high volatility, they are evaluated with a shorter look-back window and prediction horizon.**

| Dataset | Features | Frequency | Samples | Look-back | Horizon |
|---|---|---|---|---|---|
| NP | 3 | 1 Hour | 52,179 | 168 | 24 |
| PJM | 3 | 1 Hour | 52,179 | 168 | 24 |
| BE | 3 | 1 Hour | 52,179 | 168 | 24 |
| FR | 3 | 1 Hour | 52,179 | 168 | 24 |
| DE | 3 | 1 Hour | 52,179 | 168 | 24 |

where $e$ is the current epoch and $E_{warmup}$ is the warm-up period. The total loss function $\mathcal{L}_{total}$ is defined as:

$$\mathcal{L}_{total} = \lambda_p \mathcal{L}_{pred} + \frac{1 - \lambda_p}{2} r(e)(\mathcal{L}_{feat} + \mathcal{L}_{afd}) \tag{17}$$

Here, $\lambda_p$ is a fixed hyperparameter balancing the prediction task, while the distillation terms are gradually introduced, allowing the student to first learn basic patterns before aligning its features with the teacher's complex representations.

## IV. EXPERIMENTS

To extensively evaluate the efficacy and robustness of our proposed framework, we conducted a series of experiments across both long-term and short-term TSF tasks.

### A. EXPERIMENTAL SETUP

#### 1) DATASETS

We conducted experiments on a comprehensive collection of real-world datasets spanning diverse domains, including energy, weather, and economics. For long-term forecasting, we selected six widely recognized benchmarks: four Electricity Transformer Temperature (ETT) datasets (ETTh1, ETTh2, ETTm1, ETTm2), the Weather dataset, and the Exchange rate dataset [21]. For short-term forecasting, to validate the generalizability of our model, we employed five electricity price datasets (NP, PJM, BE, FR, DE) collected from major power markets over a six-year period [39]. The statistics of these datasets are summarized in Table 1 and Table 2.

#### 2) BASELINE MODELS

To validate the performance of our proposed method, we conducted a comparative analysis against a comprehensive set of state-of-the-art baselines. We utilized the

official source codes for all baseline experiments to ensure fair reproducibility. The baselines are categorized by their architectural paradigms:

- **MLP/CNN-based Models:** We selected **DLinear** [9] and **TimeMixer** [29], representative lightweight models recognized for their efficiency in capturing temporal trends and seasonality.
- **Transformer-based Models:** We included **PatchTST** [11], **iTransformer** [23], and **TimeXer** [40]. These models leverage self-attention mechanisms and currently serve as the standard benchmarks for long-term forecasting accuracy.
- **LLM-based Models:** To evaluate the effectiveness of knowledge distillation from large foundation models, we compared our student model against **AutoTimes** [6] and the recent foundation model **Timer-XL** [41]. These models leverage massive pre-trained knowledge, representing a strong performance baseline for large-scale models.

#### 3) EVALUATION METRICS

We adopted Mean Squared Error (MSE) and Mean Absolute Error (MAE) to quantitatively evaluate forecasting accuracy. MSE is widely used to assess the model's robustness by penalizing larger errors, while MAE measures the average magnitude of errors.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \quad MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{18}$$

where $y$ is the ground truth and $\hat{y}$ is the predicted value. Lower values indicate better performance for both metrics.

#### 4) IMPLEMENTATION DETAILS

Our proposed framework was implemented using PyTorch. The experiments were conducted on a server equipped with a single NVIDIA A100 (40GB) GPU.

- **Model Hyperparameters:** We set the embedding dimension $D_{LLM}$ of the teacher model to 4096 and the hidden dimension $D$ of the student model to 512. The distillation balancing coefficient $\lambda_p$ in Eq. 17 was fixed at 0.4. Additionally, the warm-up function $r(e)$ was designed to linearly increase from 0 to 1 over the entire training course (10 epochs), thereby progressively emphasizing the distillation objective and stabilizing the training.
- **Teacher Model Setup:** The teacher model employs the pre-trained Llama-2-7B [42] as a backbone. We bypass the native tokenizer and use a linear projection to map numerical patches ($P = 96$) into the embedding space ($D_{LLM} = 4096$). The resulting features are projected to the student's dimension ($D = 512$) and subsequently refined by a 1-layer Transformer encoder. While the LLM parameters were frozen to retain generalization capabilities, the projection layers and the Transformer encoder were optimized during training.
- **Student Model Setup:** The student model is composed of a stack of 3 GDC blocks with kernel size $k = 3$ and exponentially increasing dilation rates $d = \{1, 2, 4\}$ to expand the receptive field efficiently.
- **Training Protocols:** For all datasets, we used a look-back window of $H = 672$ for long-term forecasting and $H = 168$ for short-term forecasting. The prediction horizons were set to {96, 192, 336, 720} for long-term tasks and {24} for short-term tasks. Our model and AutoTimes were trained solely on a fixed prediction horizon of $P = 96$. For longer forecasting horizons ($P \in \{192, 336, 720\}$), predictions were generated via autoregressive rolling forecasting, where the model iteratively predicts the next 96 steps using its own prior outputs. In contrast, all other baseline models were trained individually for each specific prediction horizon (i.e., separate models were trained for 96, 192, 336, and 720) following their standard protocols. Despite the disadvantage of error accumulation in rolling forecasting, this single-model approach is essential for edge devices where storing multiple model weights is infeasible. Consequently, our results demonstrate that a single fixed-length model can effectively handle diverse prediction horizons in resource-constrained environments. The model was trained for 10 epochs with a batch size of 128, using the Adam optimizer with a learning rate of $1 \times 10^{-4}$.

### B. MAIN RESULTS

To validate the effectiveness of our proposed framework, we conducted a comprehensive evaluation on both long-term and short-term forecasting tasks. We compared the proposed model, denoted as GDC (w/ KD), against a diverse set of state-of-the-art baselines, ranging from lightweight linear models to large-scale LLM-based forecasters. Additionally, we report the performance of the student model trained without distillation, denoted as GDC (w/o KD), to isolate the contribution of the proposed architectural design.

#### 1) LONG-TERM TIME SERIES FORECASTING

Following the standard protocol in time series forecasting research, we evaluated the models on multivariate long-term forecasting tasks across four prediction horizons: $H \in \{96, 192, 336, 720\}$. Table 3 presents the performance comparison on long-term TSF datasets.

The results demonstrate that the proposed distillation framework significantly enhances forecasting accuracy. Specifically, our GDC (w/ KD) model achieves an average MSE reduction of 11.5% and MAE reduction of 7.8% compared to the aggregated average of baseline models. This challenges the prevailing assumption that larger parameter counts inherently guarantee superior performance in time series tasks.

Moreover, the performance gap between GDC (w/ KD) and GDC (w/o KD) validates the efficacy of our distillation framework. Although the standalone student model already

**TABLE 3. Long-term time series forecasting results. We compare the proposed method against state-of-the-art baselines across four prediction horizons (96, 192, 336, 720). The best results are highlighted in bold and the second best are underlined. MSE and MAE are used as metrics (lower is better).**

| Method | | Proposed w/ KD | | Proposed w/o KD | | Timer-XL | | AutoTimes | | TimeXer | | iTransformer | | PatchTST | | DLinear | | TimeMixer | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Metric | | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| ETTh1 | 96 | 0.362 | 0.393 | 0.372 | 0.403 | **0.358** | **0.390** | 0.360 | 0.400 | 0.383 | 0.414 | 0.387 | 0.419 | 0.398 | 0.420 | 0.369 | 0.400 | 0.403 | 0.432 |
| | 192 | 0.402 | **0.416** | 0.413 | 0.428 | 0.401 | **0.416** | **0.388** | 0.419 | 0.416 | 0.436 | 0.422 | 0.443 | 0.444 | 0.454 | 0.403 | 0.420 | 0.438 | 0.455 |
| | 336 | 0.425 | **0.429** | 0.444 | 0.445 | 0.421 | 0.430 | **0.401** | **0.429** | 0.458 | 0.464 | 0.449 | 0.463 | 0.499 | 0.487 | 0.436 | 0.446 | 0.439 | 0.456 |
| | 720 | 0.431 | 0.449 | 0.498 | 0.478 | 0.432 | 0.457 | **0.406** | **0.440** | 0.504 | 0.510 | 0.547 | 0.534 | 0.561 | 0.530 | 0.475 | 0.497 | 0.977 | 0.695 |
| | avg | 0.405 | **0.422** | 0.432 | 0.439 | 0.403 | 0.423 | **0.389** | **0.422** | 0.440 | 0.456 | 0.451 | 0.465 | 0.476 | 0.473 | 0.421 | 0.441 | 0.564 | 0.510 |
| ETTh2 | 96 | **0.275** | **0.337** | 0.283 | 0.344 | 0.289 | 0.345 | 0.295 | 0.358 | 0.282 | 0.345 | 0.304 | 0.362 | 0.331 | 0.384 | 0.305 | 0.371 | 0.313 | 0.373 |
| | 192 | **0.335** | **0.379** | 0.360 | 0.395 | 0.349 | 0.389 | 0.366 | 0.406 | 0.361 | 0.399 | 0.380 | 0.408 | 0.433 | 0.436 | 0.404 | 0.433 | 0.379 | 0.407 |
| | 336 | **0.361** | **0.402** | 0.395 | 0.426 | 0.364 | 0.411 | 0.398 | 0.436 | 0.383 | 0.420 | 0.433 | 0.445 | 0.441 | 0.446 | 0.542 | 0.511 | 0.395 | 0.440 |
| | 720 | **0.393** | **0.431** | 0.451 | 0.469 | 0.419 | 0.450 | 0.450 | 0.474 | 0.427 | 0.455 | 0.482 | 0.488 | 0.427 | 0.455 | 0.822 | 0.642 | 0.455 | 0.475 |
| | avg | **0.341** | **0.387** | 0.372 | 0.409 | 0.355 | 0.399 | 0.377 | 0.419 | 0.363 | 0.405 | 0.400 | 0.426 | 0.408 | 0.430 | 0.518 | 0.489 | 0.386 | 0.424 |
| ETTm1 | 96 | 0.290 | 0.343 | **0.288** | **0.340** | 0.289 | 0.346 | 0.294 | 0.347 | 0.315 | 0.365 | 0.313 | 0.368 | 0.300 | 0.357 | 0.307 | 0.350 | 0.321 | 0.362 |
| | 192 | **0.331** | 0.368 | **0.331** | **0.367** | 0.336 | 0.376 | 0.341 | 0.375 | 0.353 | 0.388 | 0.347 | 0.386 | 0.361 | 0.393 | 0.337 | 0.369 | 0.364 | 0.395 |
| | 336 | 0.366 | 0.388 | **0.365** | 0.388 | 0.377 | 0.402 | 0.376 | 0.398 | 0.381 | 0.405 | 0.380 | 0.408 | 0.388 | 0.413 | 0.366 | **0.387** | 0.435 | 0.435 |
| | 720 | 0.424 | 0.421 | 0.422 | 0.419 | 0.455 | 0.447 | 0.430 | 0.434 | 0.439 | 0.439 | 0.450 | 0.439 | 0.476 | 0.463 | **0.415** | **0.414** | 0.504 | 0.475 |
| | avg | 0.353 | 0.380 | **0.352** | **0.379** | 0.364 | 0.393 | 0.360 | 0.389 | 0.372 | 0.399 | 0.372 | 0.403 | 0.381 | 0.407 | 0.356 | 0.380 | 0.406 | 0.417 |
| ETTm2 | 96 | 0.165 | **0.250** | 0.171 | 0.254 | 0.175 | 0.258 | 0.180 | 0.269 | 0.176 | 0.261 | 0.180 | 0.274 | 0.189 | 0.274 | **0.163** | 0.258 | 0.175 | 0.260 |
| | 192 | 0.225 | **0.290** | 0.232 | 0.294 | 0.240 | 0.302 | 0.236 | 0.307 | 0.245 | 0.311 | 0.243 | 0.316 | 0.235 | 0.305 | **0.219** | 0.298 | 0.237 | 0.306 |
| | 336 | **0.277** | **0.326** | 0.282 | 0.330 | 0.296 | 0.340 | 0.288 | 0.340 | 0.313 | 0.351 | 0.297 | 0.350 | 0.289 | 0.342 | **0.277** | 0.341 | 0.301 | 0.343 |
| | 720 | **0.364** | **0.383** | 0.367 | 0.389 | 0.375 | 0.391 | 0.367 | 0.389 | 0.378 | 0.405 | 0.375 | 0.401 | 0.404 | 0.422 | 0.410 | 0.434 | 0.371 | 0.392 |
| | avg | **0.258** | **0.312** | 0.263 | 0.317 | 0.272 | 0.323 | 0.268 | 0.326 | 0.278 | 0.332 | 0.274 | 0.335 | 0.279 | 0.336 | 0.267 | 0.333 | 0.271 | 0.325 |
| Weather | 96 | **0.145** | **0.192** | 0.147 | 0.193 | 0.159 | 0.209 | 0.157 | 0.207 | 0.158 | 0.211 | 0.172 | 0.224 | 0.158 | 0.213 | 0.168 | 0.227 | 0.147 | 0.200 |
| | 192 | **0.190** | **0.236** | 0.192 | **0.236** | 0.207 | 0.256 | 0.205 | 0.253 | 0.202 | 0.250 | 0.222 | 0.265 | 0.214 | 0.270 | 0.211 | 0.267 | 0.199 | 0.246 |
| | 336 | **0.240** | 0.276 | 0.241 | **0.275** | 0.262 | 0.299 | 0.260 | 0.296 | 0.251 | 0.291 | 0.288 | 0.309 | 0.249 | 0.289 | 0.256 | 0.304 | 0.250 | 0.288 |
| | 720 | **0.314** | 0.329 | **0.314** | **0.327** | 0.332 | 0.350 | 0.332 | 0.346 | 0.316 | 0.338 | 0.362 | 0.360 | 0.318 | 0.340 | 0.317 | 0.356 | 0.324 | 0.338 |
| | avg | **0.222** | **0.258** | 0.224 | **0.258** | 0.240 | 0.279 | 0.239 | 0.276 | 0.232 | 0.273 | 0.261 | 0.290 | 0.235 | 0.278 | 0.238 | 0.289 | 0.230 | 0.268 |
| Exchange | 96 | **0.084** | **0.205** | 0.091 | 0.211 | 0.093 | 0.212 | 0.108 | 0.234 | 0.139 | 0.266 | 0.123 | 0.257 | 0.100 | 0.225 | 0.124 | 0.269 | 0.092 | 0.219 |
| | 192 | 0.182 | 0.303 | 0.193 | 0.314 | 0.199 | 0.318 | 0.238 | 0.347 | 0.262 | 0.371 | 0.218 | 0.337 | 0.221 | 0.344 | 0.222 | 0.356 | **0.171** | **0.293** |
| | 336 | **0.342** | **0.425** | 0.353 | 0.433 | 0.372 | 0.438 | 0.485 | 0.504 | 0.469 | 0.506 | 0.372 | 0.455 | 0.357 | 0.439 | 0.522 | 0.552 | 0.377 | 0.452 |
| | 720 | 0.874 | 0.710 | **0.853** | **0.700** | 0.961 | 0.730 | 1.314 | 0.858 | 1.214 | 0.835 | 1.031 | 0.776 | 1.159 | 0.817 | 1.166 | 0.815 | 1.108 | 0.786 |
| | avg | **0.371** | **0.411** | 0.373 | 0.415 | 0.406 | 0.425 | 0.536 | 0.486 | 0.521 | 0.495 | 0.436 | 0.456 | 0.459 | 0.456 | 0.509 | 0.498 | 0.437 | 0.438 |

exhibits competitive performance due to the efficient GDC blocks—surpassing DLinear and matching PatchTST in several cases—the application of KD further boosts performance. This indicates that the student successfully learns to pay attention to critical temporal features that are difficult to capture with a pure CNN architecture. Remarkably, our model achieves these results using a single fixed-horizon model via rolling forecasting, surpassing baselines that require separate models for each horizon. This confirms its robustness against error accumulation and suitability for memory-constrained edge deployment.

#### 2) SHORT-TERM TIME SERIES FORECASTING

To verify the robustness of our model in scenarios characterized by high volatility and rapid fluctuations, we evaluated performance on five short-term electricity price datasets (NP, PJM, BE, FR, DE) with a prediction horizon of 24 hours. The results are summarized in Table 4.

The proposed model demonstrates superior performance in short-term forecasting, achieving average MSE and MAE reductions of 12.4% and 10.9%, respectively, compared to the baselines. These results indicate that the proposed framework is effective not only in modeling long-term global temporal patterns but also in capturing short-term local features.

### C. EFFICIENCY ANALYSIS

A key objective of this study is to effectively address the trade-off between model complexity and predictive accuracy.

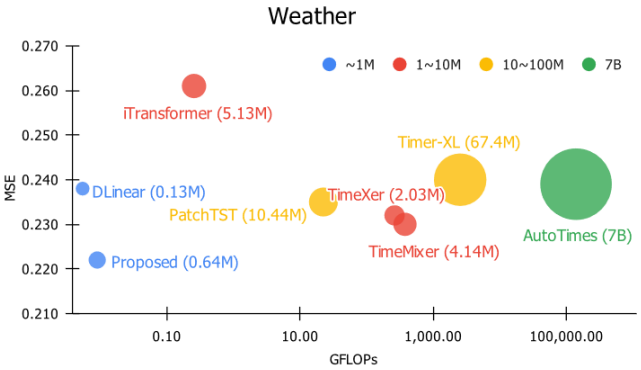



**FIGURE 8. Efficiency vs. performance on weather dataset. The x-axis denotes GFLOPs (log scale), the y-axis denotes MSE, and circle size represents parameter count.**

Figure 8 visualizes the relationship between model size (parameters), computational cost (GFLOPs), and forecasting error (MSE) on the Weather dataset.

Our model is positioned at the efficient frontier (bottom-left corner), indicating high accuracy with minimal resource usage. The proposed model demonstrates exceptional efficiency by utilizing only 0.64M parameters, representing approximately 0.01% of the scale of LLM-based models like AutoTimes (7B). Despite this massive reduction in model size, it achieves a superior MSE of 0.222 compared to 0.240 for AutoTimes, effectively challenging the prevailing

**TABLE 4. Short-term time series forecasting results on electricity price datasets. The best results are highlighted in bold and the second best are underlined. MSE and MAE are used as metrics (lower is better).**

| Methods | Proposed w/ KD | | Proposed w/o KD | | Timer-XL | | AutoTimes | | TimeXer | | iTransformer | | PatchTST | | DLinear | | TimeMixer | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| NP | **0.245** | **0.266** | 0.251 | 0.271 | 0.246 | 0.276 | 0.287 | 0.302 | **0.245** | 0.271 | 0.260 | 0.297 | 0.278 | 0.294 | 0.297 | 0.313 | 0.294 | 0.310 |
| PJM | **0.083** | **0.182** | 0.085 | **0.182** | 0.103 | 0.203 | 0.105 | 0.202 | 0.094 | 0.194 | 0.101 | 0.198 | 0.115 | 0.217 | 0.100 | 0.210 | 0.114 | 0.223 |
| BE | **0.374** | 0.247 | 0.375 | 0.245 | 0.477 | 0.306 | 0.441 | 0.298 | 0.379 | **0.242** | 0.417 | 0.282 | 0.408 | 0.265 | 0.450 | 0.310 | 0.424 | 0.278 |
| FR | **0.370** | 0.192 | 0.375 | **0.190** | 0.496 | 0.245 | 0.488 | 0.269 | 0.388 | 0.210 | 0.444 | 0.244 | 0.415 | 0.230 | 0.403 | 0.247 | 0.420 | 0.229 |
| DE | 0.435 | 0.415 | 0.439 | 0.417 | **0.428** | **0.408** | 0.532 | 0.456 | 0.453 | 0.420 | 0.474 | 0.444 | 0.493 | 0.445 | 0.504 | 0.455 | 0.473 | 0.443 |
| AVG | **0.301** | **0.260** | 0.305 | 0.261 | 0.350 | 0.288 | 0.371 | 0.305 | 0.312 | 0.267 | 0.339 | 0.293 | 0.342 | 0.290 | 0.351 | 0.307 | 0.345 | 0.297 |

"bigger is better" scaling law in time series forecasting. Furthermore, with a computational cost of merely 0.009 GFLOPs, the model demands orders of magnitude fewer resources than Transformer-based baselines such as iTransformer (5.13M params) and PatchTST (10.44M params). Although DLinear offers a lower parameter count (0.13M), it exhibits significantly higher error rates (MSE 0.240); thus, our model achieves substantial performance gains with only a marginal increase in complexity, offering a more practical and optimal balance for deployment in resource-constrained edge environments.

These results confirm that the heterogeneous distillation framework successfully transfers the complex modeling capabilities of heavy models into a compact architecture, solving the efficiency-accuracy trade-off.

## V. ANALYSIS AND DISCUSSION

In this section, we conduct a comprehensive ablation study to validate the individual contributions of the proposed components. Furthermore, we demonstrate the practical feasibility of our framework by detailing the deployment of the distilled model on a resource-constrained MCU.

### A. ABLATION STUDIES

To validate the effectiveness of our proposed framework, we conducted comprehensive ablation studies focusing on two critical aspects: the contribution of different knowledge distillation loss components and the architectural design of the student model's GDC block.

#### 1) EFFECTIVENESS OF KNOWLEDGE DISTILLATION STRATEGY

We investigated the individual contributions of the distillation objectives to verify whether transferring attention-level knowledge is as crucial as transferring feature-level representations. The variants evaluated are as follows:

- **w/o AFD ($\mathcal{L}_{feat}$ only):** This variant excludes the AFD loss, relying solely on global feature alignment. It tests the necessity of the attention-guided masking mechanism.
- **w/o FD ($\mathcal{L}_{afd}$ only):** This variant removes the general feature distillation loss, utilizing only the AFD loss to assess if focusing solely on salient regions is sufficient.
- **w/o KD (Student only):** This baseline trains the student model exclusively with the prediction loss $\mathcal{L}_{pred}$, serving as a reference to quantify the gain achieved by our heterogeneous distillation framework.

#### 2) EFFECTIVENESS OF GDC BLOCK ARCHITECTURE

To justify the design of the GDC block, we eliminated its key components:

- **w/o DSC:** Replaces DSC with standard convolutions to evaluate parameter efficiency.
- **w/o Dilated Conv:** Removes the dilation factor (setting $d = 1$) to verify the importance of expanding the receptive field for long-term dependency.
- **w/o GLU:** Substitutes the GLU activation function with a standard ReLU activation function to assess the efficacy of the dynamic gating mechanism in feature selection.

Table 5 summarizes the results on the long-term TSF datasets.

The quantitative results in Table 5 demonstrate that the combination of $\mathcal{L}_{feat}$ and $\mathcal{L}_{afd}$ yields the lowest error rates. Notably, removing the AFD mechanism (*w/o AFD*) leads to performance degradation, particularly in complex datasets. This corroborates our hypothesis that simply aligning feature maps globally is insufficient for heterogeneous distillation; the student must explicitly learn *where* to focus. The teacher's attention maps successfully act as a spatial regularizer, guiding the student to prioritize temporally salient patches. Furthermore, the significant gap between the *w/o KD* baseline and the distilled variants confirms that the lightweight CNN student inherently lacks the capacity to capture complex long-term dependencies from data alone, a limitation effectively mitigated by transferring privileged knowledge from the Transformer-based teacher.

The ablation of the GDC block reveals that each component plays an imperative role. First, removing dilated convolutions (*w/o Dilated Conv*) results in a notable increase in MSE on the Weather dataset. This underscores the necessity of an expanded receptive field for capturing multi-scale temporal patterns in long sequences. Second, replacing the GLU with ReLU (*w/o GLU*) degrades performance, indicating that the gating mechanism is critical for suppressing noise and selectively propagating relevant features. Finally, while removing DSC (*w/o DSC*) might theoretically increase model capacity, it paradoxically leads to higher errors, likely due to overfitting or optimization difficulties given the limited data regime. Thus, the synergistic integration of DSC,

**TABLE 5.** Ablation study of key components. We analyze the impact of different distillation losses and architectural choices on forecasting performance.

| Methods | ETTh1 | | ETTh2 | | ETTm1 | | ETTm2 | | Exchange | | Weather | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE | MSE | MAE |
| **Proposed w/ KD** | **0.405** | **0.422** | **0.341** | **0.387** | 0.353 | 0.380 | **0.258** | 0.312 | **0.371** | **0.411** | **0.222** | 0.258 |
| *Distillation Ablation* | | | | | | | | | | | | |
| w/o AFD ($\mathcal{L}_{feat}$ only) | 0.407 | **0.422** | **0.341** | **0.387** | 0.353 | 0.380 | 0.259 | 0.313 | 0.393 | 0.430 | **0.222** | 0.259 |
| w/o FD ($\mathcal{L}_{afd}$ only) | 0.411 | 0.423 | 0.343 | 0.388 | 0.351 | **0.378** | **0.258** | **0.311** | 0.398 | 0.425 | **0.222** | **0.256** |
| w/o KD (student only) | 0.432 | 0.439 | 0.372 | 0.409 | 0.352 | 0.379 | 0.263 | 0.317 | 0.373 | 0.415 | 0.224 | 0.258 |
| *Architecture Ablation* | | | | | | | | | | | | |
| w/o DSC | 0.427 | 0.438 | 0.379 | 0.410 | 0.353 | 0.381 | 0.263 | 0.317 | 0.444 | 0.448 | 0.223 | 0.257 |
| w/o Dilated Conv | 0.426 | 0.432 | 0.351 | 0.393 | 0.357 | 0.383 | 0.259 | 0.313 | 0.393 | 0.423 | 0.230 | 0.264 |
| w/o GLU | 0.421 | 0.435 | 0.366 | 0.403 | **0.350** | 0.382 | 0.274 | 0.321 | 0.392 | 0.424 | 0.226 | 0.262 |

**TABLE 6.** Performance and efficiency comparison between FP32 and INT8 precision on the ETTh1 dataset. Inference speed is measured at a prediction horizon of 96.

| Precision | MSE at Prediction Horizon ($T$) | | | | Speed (ms) |
|---|---|---|---|---|---|
| | 96 | 192 | 336 | 720 | |
| **FP32 (Proposed)** | **0.362** | **0.402** | **0.425** | **0.431** | **256** |
| INT8 (Dynamic) | 0.363 | 0.405 | 0.432 | 0.458 | 416 |

**TABLE 7.** Detailed resource usage on ESP32-S3 with input sequence length $H = 672$. SRAM usage is broken down into the tensor arena (runtime inference memory), I/O buffers, and system overheads.

| Metric | Value |
|---|---|
| **Model Size (Flash)** | 737 KB |
| **Total SRAM Usage** | $\approx$ 201 KB |
| - Tensor Arena (Peak Runtime) | 175 KB |
| - I/O Buffers & Queues | 11 KB |
| - FreeRTOS Task Stacks | 15 KB |
| **Inference Latency** | 256 ms |

dilation, and GLU is optimal for balancing efficiency and expressiveness.

### B. ON-DEVICE DEPLOYMENT

To verify the real-world applicability of our model in edge computing scenarios, we deployed the trained student model on the ESP32-S3 MCU.

#### 1) HARDWARE SPECIFICATION AND SETUP

The ESP32-S3 features a dual-core Xtensa LX7 CPU running at 240 MHz with 512 KB of internal SRAM and 8 MB of PSRAM. This environment represents a typical constraint for edge AI, where memory and computational power are severely limited compared to server-grade GPUs. We implemented the inference engine using TensorFlow Lite for Microcontrollers (TFLM).

#### 2) IMPLEMENTATION STRATEGY: PRECISION VS. EFFICIENCY

A critical design decision in this deployment was the choice of numerical precision. While 8-bit integer quantization (INT8) is standard practice in TinyML applications, such as image classification, to reduce model size and latency, it presents challenges for TSF, which is fundamentally a regression problem dealing with continuous numerical values. In regression tasks, fine-grained precision is paramount, as quantization errors can accumulate through temporal layers and potentially distort signal trends. As demonstrated in Table 6, applying dynamic INT8 quantization progressively degrades predictive accuracy at longer horizons, increasing the MSE from 0.431 to 0.458 at $T = 720$. Furthermore, the computational overhead of dynamic quantization unexpectedly increased inference latency from 256 ms (FP32) to 416 ms. This delay primarily stems from a severe type-casting bottleneck caused by maintaining FP32 precision during the patching and depatching stages. Consequently, to ensure that the high predictive fidelity and efficiency achieved during training are fully preserved in the embedded environment, we strategically decided to maintain 32-bit floating-point (FP32) precision without applying quantization.

#### 3) DEPLOYMENT RESULTS

Table 7 details the on-device resource consumption breakdown. The total SRAM usage is approximately 201 KB, predominantly occupied by the Tensor Arena (175 KB). The Tensor Arena functions as a contiguous memory pool for the TensorFlow Lite Micro interpreter, storing intermediate tensors and activation maps during inference. To prevent memory fragmentation, a static block of 200 KB was allocated in the internal SRAM, implying a utilization rate of 87.5% (175 KB). The remaining overhead consists of 11 KB for floating-point I/O buffers ($H = 672$) and 15 KB dedicated to the stacks of three FreeRTOS tasks (Inference task, result send task, and data receive task). Regarding timing, the inference latency is measured at 256 ms. Considering that typical industrial monitoring applications (e.g., temperature or load forecasting) operate with sampling intervals in the order of seconds or minutes, this sub-second latency falls well within the requirements for real-time edge processing.

## VI. CONCLUSION

In this work, we proposed a novel heterogeneous knowledge distillation framework to address the challenge of deploying high-performance forecasting models on resource-constrained devices. By integrating the efficient GDC Block with the AFD loss, our approach enables a lightweight CNN student to inherit the complex temporal reasoning of a Transformer teacher without incurring prohibitive computational costs.

Our extensive experiments on both long-term and short-term forecasting benchmarks demonstrated that the proposed model achieves state-of-the-art performance. Notably, it outperformed the LLM-based baseline, AutoTimes, reducing the Mean Squared Error (MSE) by approximately 10% while utilizing less than 0.01% of the parameters. This result empirically challenges the prevailing ''scaling law'' assumption in time series forecasting, proving that domain-specific lightweight architectures, when properly guided by high-capacity teachers, can surpass general-purpose foundation models in both accuracy and efficiency.

Furthermore, we validated the practical feasibility of our approach through on-device deployment. The distilled model was successfully implemented on an ESP32-S3 MCU using full FP32 precision, achieving real-time inference with a latency of 256 ms and a peak memory usage of only 175 KB. This confirms that our framework serves as a viable solution for bringing server-grade predictive intelligence to the extreme edge, enabling applications in smart factories, energy management, and IoT sensor networks where connectivity and hardware resources are limited.

Despite these achievements, future research is needed to deepen the theoretical understanding of feature alignment dynamics between heterogeneous architectures. Additionally, extending this framework to broader time series tasks, such as anomaly detection, and verifying its robustness against real-world sensor noise and concept drift remain critical steps for wider industrial adoption.

## REFERENCES

[1] D. A. Tedjopurnomo, Z. Bao, B. Zheng, F. M. Choudhury, and A. K. Qin, ''A survey on modern deep neural network for traffic prediction: Trends, methods and challenges,'' *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 4, pp. 1544–1561, Apr. 2022.

[2] I. Amalou, N. Mouhni, and A. Abdali, ''Multivariate time series prediction by RNN architectures for energy consumption forecasting,'' *Energy Rep.*, vol. 8, pp. 1084–1091, Nov. 2022.

[3] P. Kamat and R. Sugandhi, ''Anomaly detection for predictive maintenance in Industry 4.0—A survey,'' *E3S Web Conf.*, vol. 170, 2020, Art. no. 02007.

[4] M. H. ur Rehman, I. Yaqoob, K. Salah, M. Imran, P. P. Jayaraman, and C. Perera, ''The role of big data analytics in industrial Internet of Things,'' *Future Gener. Comput. Syst.*, vol. 99, pp. 247–259, Oct. 2019.

[5] J. Kim, H. Kim, H. Kim, D. Lee, and S. Yoon, ''A comprehensive survey of deep learning for time series forecasting: Architectural diversity and open challenges,'' *Artif. Intell. Rev.*, vol. 58, no. 7, Apr. 2025, Art. no. 216.

[6] Y. Liu, G. Qin, X. Huang, J. Wang, and M. Long, ''AutoTimes: Autoregressive time series forecasters via large language models,'' in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 12215–12236.

[7] M. Jin, ''Time-LLM: Time series forecasting by reprogramming large language models,'' in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[8] F. Shi, X. Yin, K. Wang, W. Tu, Q. Sun, and H. Ning, ''Large language models for time series analysis: Techniques, applications, and challenges,'' 2025, *arXiv:2506.11040*.

[9] A. Zeng, M. Chen, L. Zhang, and Q. Xu, ''Are transformers effective for time series forecasting?'' in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 9, pp. 11121–11128.

[10] C. Liu, H. Miao, Q. Xu, S. Zhou, C. Long, Y. Zhao, Z. Li, and R. Zhao, ''Efficient multivariate time series forecasting via calibrated language models with privileged knowledge distillation,'' 2025, *arXiv:2505.02138*.

[11] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, ''A time series is worth 64 words: Long-term forecasting with transformers,'' in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[12] Z. Hao, ''One-for-all: Bridge the gap between heterogeneous architectures in knowledge distillation,'' in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 79570–79582.

[13] F. Yu and V. Koltun, ''Multi-scale context aggregation by dilated convolutions,'' in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[14] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, ''MobileNets: Efficient convolutional neural networks for mobile vision applications,'' 2017, *arXiv:1704.04861*.

[15] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, ''Language modeling with gated convolutional networks,'' in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 70, 2017, pp. 933–941.

[16] G. E. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed., Hoboken, NJ, USA: Wiley, 2015.

[17] R. J. Hyndman, A. B. Koehler, J. K. Ord, and R. D. Snyder, *Forecasting With Exponential Smoothing: The State Space Approach*. Berlin, Germany: Springer, 2008.

[18] S. Hochreiter and J. Schmidhuber, ''Long short-term memory,'' *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[19] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, ''Learning phrase representations using RNN encoder–decoder for statistical machine translation,'' in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.

[20] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, ''Informer: Beyond efficient transformer for long sequence time-series forecasting,'' in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 12, pp. 11106–11115.

[21] H. Wu, J. Xu, J. Wang, and M. Long, ''Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,'' in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 22419–22430.

[22] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, ''FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting,'' in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 27268–27286.

[23] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, ''iTransformer: Inverted transformers are effective for time series forecasting,'' in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[24] Y. Zhang, ''A survey on time-series pre-trained models,'' in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2024, pp. 8366–8375.

[25] A. Garza, C. Challu, and M. Mergenthaler-Canseco, ''TimeGPT-1,'' 2023, *arXiv:2310.03589*.

[26] K. Rasul, ''Lag-Llama: Towards foundation models for probabilistic time series forecasting,'' in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[27] A. Ansari, ''Chronos: Learning the language of time series,'' in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024.

[28] G. Woo, C. Liu, A. Sahoo, A. Kumar, and C. Xiong, ''Unified training of universal time series forecasters,'' in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[29] S. Wang, ''TimeMixer: Decomposable multiscale mixing for time series forecasting,'' in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[30] A. Das, ''Long-term forecasting with TiDE: Time-series dense encoder,'' *Trans. Mach. Learn. Res.*, 2024.

[31] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, ''TimesNet: Temporal 2D-variation modeling for general time series analysis,'' in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[32] M. Liu, "SCINet: Time series modeling and forecasting with sample convolution and interaction," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 5816–5828.
[33] D. Luo and X. Wang, "ModernTCN: A modern pure convolution structure for general time series analysis," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
[34] Z. Lai, D. Zhang, H. Li, C. S. Jensen, H. Lu, and Y. Zhao, "LightCTS*: Lightweight correlated time series forecasting enhanced with model distillation," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 8695–8710, Dec. 2024.
[35] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
[36] V. Vapnik and A. Vashist, "A new learning paradigm: Learning using privileged information," *Neural Netw.*, vol. 22, nos. 5–6, pp. 544–557, Jul. 2009.
[37] Q. Xu, K. Wu, M. Wu, K. Mao, X. Li, and Z. Chen, "Reinforced knowledge distillation for time series regression," *IEEE Trans. Artif. Intell.*, vol. 5, no. 6, pp. 3184–3194, Jun. 2024.
[38] J. Lin, "MCUNet: Tiny deep learning on IoT devices," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 11711–11722.
[39] J. Lago, G. Marcjasz, B. De Schutter, and R. Weron, "Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark," *Appl. Energy*, vol. 293, Jul. 2021, Art. no. 116983.
[40] J. Dong, Y. Liu, M. Long, G. Qin, Y. Qiu, J. Wang, Y. Wang, H. Wu, and H. Zhang, "TimeXer: Empowering transformers for time series forecasting with exogenous variables," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 469–498.
[41] Y. Liu, G. Qin, X. Huang, J. Wang, and M. Long, "Timer-XL: Long-context transformers for unified time series forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
[42] H. Touvron, "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.



**SANGJIN NA** was born in 1997. He received the B.S. degree from Pusan National University, in 2024, where he is currently pursuing the M.S. degree. His research interests include embedded systems, embedded AI, TinyML, and time series forecasting.



**YONGHUN CHO** was born in 1995. He received the B.S. degree from Dong-Eui University, Busan, Republic of Korea, in 2020. He is currently pursuing the Ph.D. degree with Pusan National University. His research interests include embedded systems, embedded AI, and mmWave systems.



**YUNJU BAEK** (Member, IEEE) was born in 1967. He received the Ph.D. degree in computer science from KAIST, Republic of Korea, in 1997. He was an invited Professor with KAIST, the CTO of Naver Corporation, and an Assistant Professor with Sookmyung Women's University. He is currently a Professor with the School of Computer Science and Engineering, Pusan National University. His research interests include embedded systems, RTLS systems, wireless sensor networks, embedded AI, TinyML, and time series analysis.